Text Splitting In LangChain

Turning long text into usable context

CONTENT

Text Splitters in LangChain

Text Splitting is the process of breaking large chunks of text (like articles, PDFs, HTML pages, or books) into smaller, manageable pieces (chunks) that an LLM can handle effectively.

Check demo: https://chunkviz.up.railway.app/

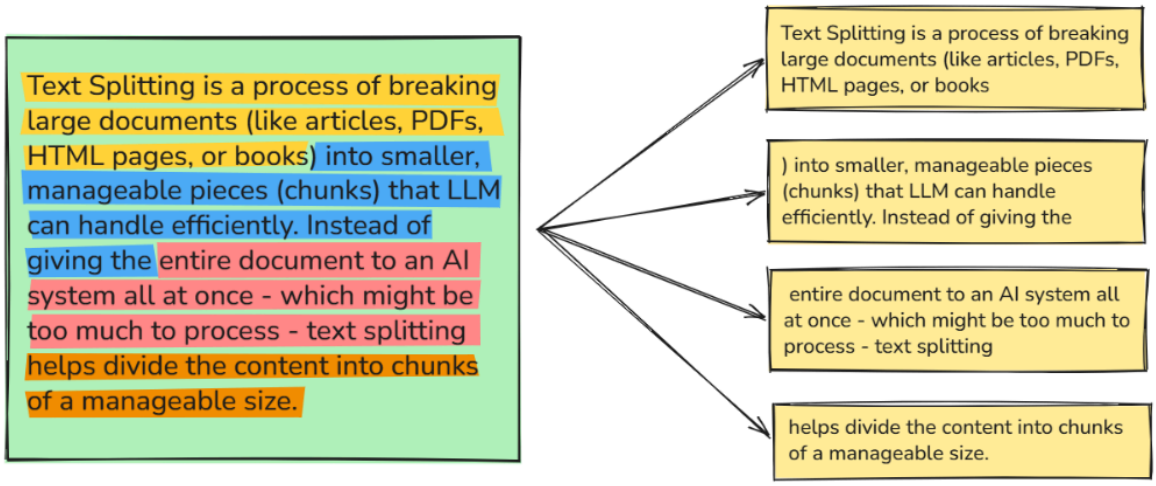
Effective text splitting helps preserve context and improves the performance of applications such as **Retrieval-Augmented Generation (RAG)**, semantic search, question answering, and document summarization. Some of the common text splitting techniques are:

- Length-Based Text Splitting
- Text Structure-Based Text Splitting
- Document Structure-Based Text Splitting
- Semantic Splitting and so on ….

Choosing the appropriate text splitter depends on the document type, model constraints, and application requirements. In this section, we will learn about different text splitting techniques.

1. Length-Based Text Splitting

Length-based splitting divides text into chunks of a specific character or token count. For **Chunk size = 100**, the text splitting look like this:




```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter

# Load document
loader = TextLoader(file_path='../1_Datasets/Harry_Potter.txt')
docs = loader.load()
```

```
# Configure splitter
splitter = CharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0,
    separator="")
```

```
# Split documents
result = splitter.split_documents(docs)
print(result[0].page_content) # printing the first splitted document
```

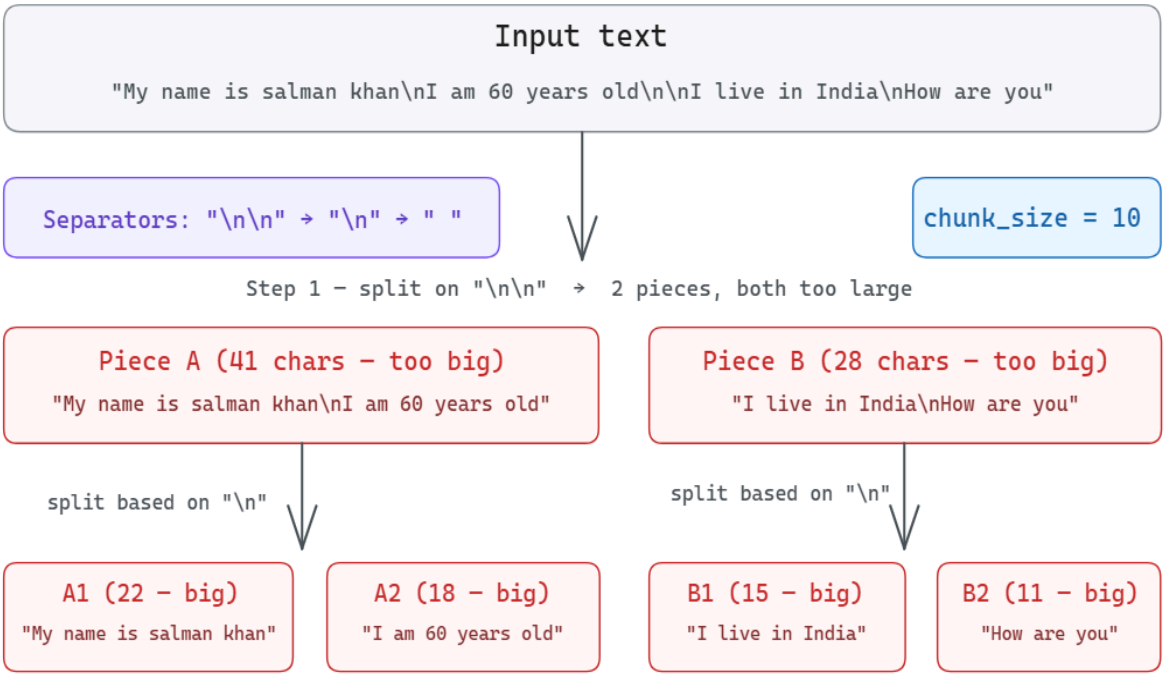
2. Text Structure-Based Splitting

A **Recursive Character Text Splitter** is a text splitting strategy that breaks large documents into smaller chunks while trying to preserve the semantic structure of the text. Instead of splitting by a **fixed number of characters**, it splits according to the natural **structure** of the document.

Splitting generally follows this pattern:

- Splitting on the basis of **Paragraph**
- Splitting on the basis of **Lines**
- Splitting on the basis of **Space**

Let's have a look on **how it works:**



Now, each sentence further splits into word level and then starts merging until the ***chunk_size*** is reached.

Line 1 — "My name is salman khan" (22 chars > 10) → split on space

Step 1: Split into individual words (by spaces)

My	name	is	salman	khan
2 ch	4 ch	2 ch	6 ch	4 ch

Step 2: Greedy merge (chunk_size = 10), trial by trial

Trial	Current Buffer (with length)	Next Word (length)	Candidate Chunk After Merge (length)	Action
1	(empty) 0	My (2)	"My" (2)	✅ Append word
2	My 2	name (4)	"My name" (7)	✅ Append word
3	My name 7	is (2)	"My name is" (10)	✅ Append word (Reached limit)
4	My name is 10	salman (6)	"My name is salman" (17)	⚠️ Boundary reached → Store current chunk: "My name is"
5	salman 6	khan (4)	"salman khan" (11)	⚠️ Boundary reached → Store current chunk: "salman"
6	khan 4	—	"khan" (4)	🏁 No more words → Store final chunk: "khan"

✅ **Result for Line 1:** Produced 3 chunks

Chunk 1	Chunk 2	Chunk 3
"My name is" (10 chars)	"salman" (6 chars)	"khan" (4 chars)

Line 2 — "I am 60 years old" (18 chars > 10) → split on space

Step 1: Split into individual words (by spaces)

"I"	"am"	"60"	"years"	"old"
1 ch	2 ch	2 ch	5 ch	3 ch

Step 2: Greedy merge (chunk_size = 10), trial by trial

Trial	Current Buffer (with length)	Next Word (length)	Candidate Chunk After Merge (length)	Action
1	(empty) 0	I (1)	"I" (1)	✅ 1 ≤ 10 → Append word
2	I 1	am (2)	"I am" (4)	✅ 4 ≤ 10 → Append word
3	I am 4	60 (2)	"I am 60" (7)	✅ 7 ≤ 10 → Append word
4	I am 60 7	years (5)	"I am 60 years" (13)	⚠️ 13 > 10 → Boundary reached Store current chunk: "I am 60"
5	years 5	old (3)	"years old" (9)	✅ End of words → Store final chunk: "years old"

✅ **Result for Line 2:** Produced 2 chunks

Chunk 1	Chunk 2
"I am 60" (7 chars)	"years old" (9 chars)

Now, let's have a look at the implementation:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """My name is salman khan
I am 60 years old

I live in India
How are you
"""

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size = 10,
    chunk_overlap = 0
)

chunks = text_splitter.split_text(text)
print(chunks)
```

```
['My name is', 'salman', 'khan', 'I am 60', 'years old', 'I live in',
'India', 'How are', 'you']
```

Adding the concept of **chunk_overlap** and applying the text splitting in previously loaded documents:

```
# let's use same splitter for bigger documents
rec_text_splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 50
)

chunks = rec_text_splitter.split_documents(docs)
print(len(chunks))
```

```
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1}")
    print(chunk.page_content)
    print("-"*30)
```

3. Document Structure-Based Splitting

Document Structure-Based Splitting is a text chunking technique where a document is divided according to its **logical structure** rather than a fixed number of characters, or sentences.

Documents are usually organized into sections, chapters, headings, paragraphs, tables, lists, and other components. Instead of splitting the text randomly, we preserve the natural boundary.

3.1 Working with Markdown Structure:

```
markdown_text = """
# Harry Potter

## Introduction
Harry Potter is a fictional character created by British author J.K.
Rowling.
He is the main protagonist of the Harry Potter book series, which
follows his journey as a young wizard.

## Early Life
Harry Potter was born to James Potter and Lily Potter. When he was a
baby, his parents were killed by the
dark wizard Lord Voldemort. Harry survived the attack and was left with
a lightning-shaped scar on his forehead.

## Hogwarts School
At the age of eleven, Harry discovers that he is a wizard and receives a
letter inviting him to attend Hogwarts
School of Witchcraft and Wizardry. There, he makes close friends:

- Ron Weasley
- Hermione Granger

## Main Characters
 - Harry Potter
 - Hermione Granger
 - Ron Weasley
 - Lord Voldemort

## Conclusion
Harry Potter is one of the most popular and influential fantasy series
in the world.
The story continues to inspire readers of all ages through its memorable
characters,
magical world, and powerful lessons. """
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter,
Language

md_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.MARKDOWN,
    chunk_size = 250,
    chunk_overlap = 0
)

chunks = md_splitter.split_text(markdown_text)
```

```
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1}")
    print(chunk)
    print("--"*30)
```

3.2 Working with Python Code:

```
python_code = """
from langchain_text_splitters import RecursiveCharacterTextSplitter,
Language

md_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.MARKDOWN,
    chunk_size = 250,
    chunk_overlap = 0
)

chunks = md_splitter.split_text(markdown_text)

for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1}")
    print(chunk)
    print("--"*30)
"""
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter,
Language

python_splitter = RecursiveCharacterTextSplitter.from_language(
    language = Language.PYTHON,
    chunk_size=145,
    chunk_overlap=0)

chunks = python_splitter.split_text(python_code)
print(len(chunks))
```

```
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1}")
    print(f"The size of chunk {i+1}: {len(chunk)}")
    print(chunk)
    print()
```

3.3 Working with JavaScript Code:

```
javascript_text = """
// Function is called, the return value will end up in x
let x = myFunction(4, 3);

function myFunction(a, b) {
// Function returns the product of a and b
  return a * b;
}
"""
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter,
Language

js_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.JS,
    chunk_size = 65,
    chunk_overlap = 0
)
chunks = js_splitter.split_text(javascript_text)
len(chunks)
```

```
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1}")
    print(f"The size of chunk {i+1}: {len(chunk)}")
    print(chunk)
    print()
```